Business Requirements Document (BRD)

Location Pockets System – Production-Ready Full-Stack Application

Version: 2.0
Date: 2025-03-02
Prepared for: Development Team

---

1. Introduction

1.1 Purpose

The Location Pockets System is a web-based tool that converts geographic coordinates (latitude/longitude) into compact alphanumeric **Pocket IDs** based on a hierarchical grid. It enables users to:

- Define a custom grid (origin, cell sizes, alphabet).
- Upload branch data and compute Pocket IDs for each branch.
- Automatically find and display the nearest branch(es) to any query point (e.g., customer location) on an interactive map.
- Visualise grid cells, branch locations, and customer dots on a map with professional layout.

This system will be built as a **production-ready full-stack application** using React (frontend) and Node.js (backend) with a database, ensuring scalability, security, and offline capability when deployed locally.

1.2 Scope

- **In scope:**
  – Configuration of grid parameters (origin, meters per degree, grid levels, 30-character alphabet).
  – Excel upload of branch master data.
  – Pocket ID generation from latitude/longitude (single and batch).
  – Reverse geocoding (Pocket ID → center coordinates).
  – **Automatic nearest branch finder**: when a user provides a location (or clicks on the map), the system immediately displays the nearest branches on the map and in a list, without manual step-by-step searches.
  – Interactive map showing grid overlay, branch markers, and customer dots with real-time nearest branch highlighting.
  – Persistence of configuration and branch data using a database (PostgreSQL / SQLite) via Node.js API.
  – All calculations can be performed either client-side or server-side (configurable).
- **Out of scope:**
  – User authentication / multi-tenant support (may be added later).
  – Real-time collaboration.
  – Mobile apps (but responsive web).

1.3 Definitions

Term	Description
Pocket ID	Alphanumeric code (e.g., 7F-33-22-11-00-44) representing a grid cell – **no country prefix**.
Grid level	One of the hierarchical cell sizes: 500 km, 100 km, 20 km, 5 km, 1 km.
Alphabet	Mapping from index 0–29 to a character (30 unique characters).
Origin	Southwest corner of the grid system (default 8°N, 68°E).
Branch master	List of branches with ID, city, latitude, longitude.
Customer dot	A point of interest (e.g., customer location) that can be dropped on the map to trigger nearest branch lookup.

2. Functional Requirements

2.1 Configuration Module

- **FR1.1:** User can view and edit the origin latitude/longitude (default 8°N, 68°E).
- **FR1.2:** User can view and edit meters per degree for latitude (default 111,000) and longitude at origin (default 109,898.5).
- **FR1.3:** Grid levels are fixed at: **500 km, 100 km, 20 km, 5 km, 1 km** (coarse to fine). User can enable/disable levels, but sizes are predefined.
- **FR1.4:** User can define a **30-character alphabet** (0–29 mapping). Default provided but editable. Must contain 30 unique characters.
- **FR1.5:** Configuration is saved to the database via API and loaded on app start.

2.2 Branch Master Management

- **FR2.1:** User can upload an Excel (.xlsx) file containing branch data.
    - Expected columns: Branch ID, City, Latitude, Longitude.
    - File is parsed on frontend; data sent to backend for storage and processing.
- **FR2.2:** User can manually add, edit, or delete branch entries via a form.
- **FR2.3:** All branch data is stored in a database (PostgreSQL or SQLite).

- **FR2.4:** For each branch, the system computes its Pocket ID (for the finest level, 1 km) upon import or manual entry. Computation can be done on backend.
- **FR2.5:** User can export branch list (with Pocket IDs) to Excel/CSV.

2.3 Pocket ID Calculator (Single Coordinate)

- **FR3.1:** Input fields for latitude and longitude (decimal degrees).
- **FR3.2:** On clicking "Generate", the system computes:
    - Meters from origin using configured conversions.
    - Row/col indices for each grid level.
    - Pocket ID string using the 30-character alphabet (no country code).
- **FR3.3:** Result is displayed in a copy-able format, showing the full ID with level breakdown.

2.4 Reverse Lookup (Pocket ID → Center)

- **FR4.1:** Input field for a Pocket ID (e.g., 7F-33-22-11-00-44).
- **FR4.2:** On submit, the system decodes the ID to obtain:
    - Row/col indices per level.
    - Total X/Y offset of the southwest corner.
    - Center coordinates by adding half the finest cell size and converting to degrees.
- **FR4.3:** Display center latitude/longitude and optionally the four corner coordinates.

2.5 Automatic Nearest Branch Finder

- **FR5.1:** User can set a query location by:
    - Entering latitude/longitude in a form.
    - Clicking directly on the map (dropping a customer dot).
- **FR5.2:** As soon as a location is provided, the system **automatically**:
    - Calculates the great-circle distance (Haversine) from that point to all branches (using backend API for performance).
    - Identifies the nearest branch(es) – user can configure how many to show (e.g., top 5).
    - Displays the nearest branches in a list, sorted by distance.
    - Highlights those branches on the map (e.g., different marker color, popup).
    - Optionally draws a radius circle around the query point.
- **FR5.3:** The map updates in real-time without requiring a manual "search" button (though a button can also be provided).
- **FR5.4:** Summary shows the nearest branch ID, distance, and city.

2.6 Map Visualisation

- **FR6.1:** Interactive map (Leaflet) with base layer (e.g., OpenStreetMap, satellite).
- **FR6.2:** Grid overlay: user can select a grid level from a dropdown (500 km, 100 km, 20 km, 5 km, 1 km) and see the corresponding cell boundaries drawn on the map.
- **FR6.3:** Branch locations shown as markers; clicking a marker shows branch details and its Pocket ID.
- **FR6.4: Customer dots** – user can:
    - Upload a separate list of customer coordinates (Excel) which appear as dots.
    - Click on the map to drop a temporary customer dot; the system immediately shows nearest branches to that dot.
- **FR6.5:** Map can be panned/zoomed; grid cells redraw dynamically.
- **FR6.6:** Highlight a specific Pocket ID cell (e.g., from reverse lookup) on the map.
- **FR6.7:** Professional layout: map takes central focus with floating control panels (grid level selector, layer toggles, quick search, nearest results panel).

2.7 Batch Processing

- **FR7.1:** User can upload an Excel file with multiple coordinates (latitude/longitude).
- **FR7.2:** System computes Pocket IDs for each row (using backend API) and generates a downloadable Excel file with original columns plus the Pocket ID(s).
- **FR7.3:** Optional: include all levels or just the finest level.

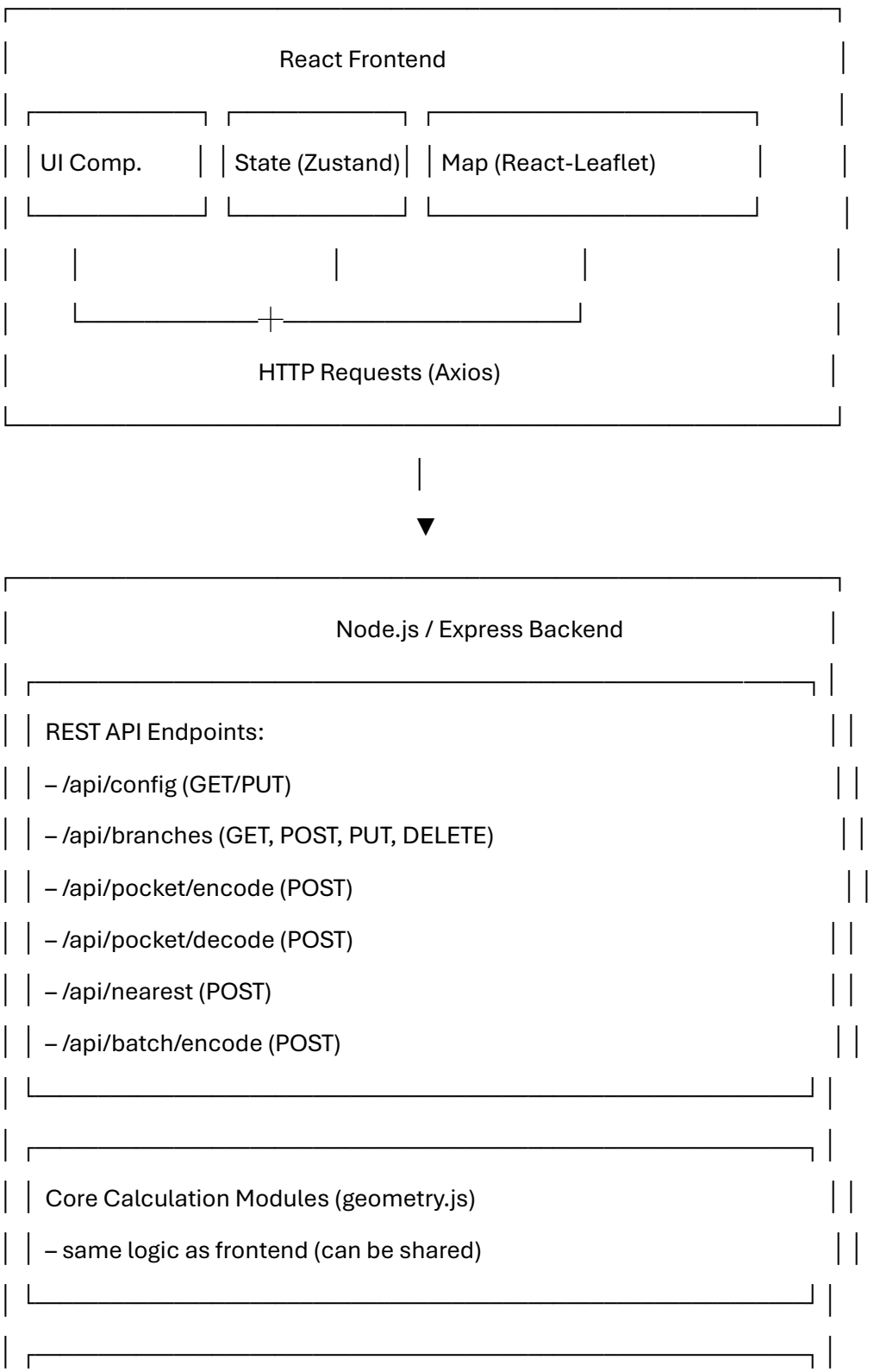
3. Non-Functional Requirements

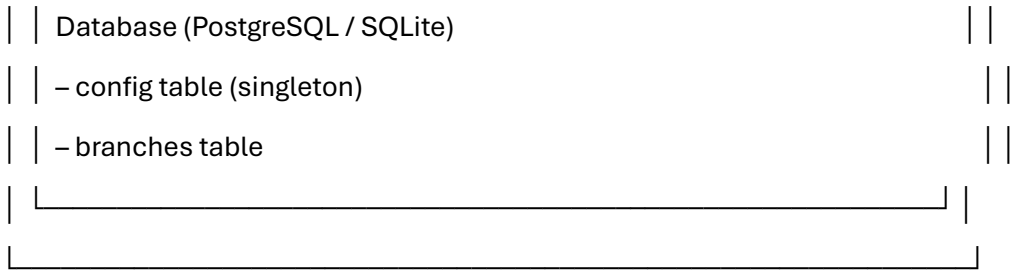
- **NFR1:** Application must be production-ready: secure, scalable, and maintainable.
- **NFR2:** Backend API built with Node.js/Express, using a database (PostgreSQL recommended, SQLite for local development).
- **NFR3:** All calculations can be performed either client-side (for speed) or server-side (for consistency); design should allow both.
- **NFR4:** Frontend must be responsive and work on desktop screens (minimum width 1280px).
- **NFR5:** Excel parsing and batch processing should not block UI; use Web Workers on frontend and asynchronous tasks on backend.
- **NFR6:** Map must support up to 1000 markers without performance degradation (use clustering if needed).
- **NFR7:** API endpoints must include proper error handling and validation.
- **NFR8:** Code should be modular, well-documented, and follow industry best practices.

4. System Architecture

The system is a **full-stack web application** with a React frontend and a Node.js/Express backend.

text





Data Persistence:

- Database stores configuration (key-value) and branch records.
- For local development, SQLite can be used; for production, PostgreSQL.

External Libraries:

- **Frontend:** React, Vite, Zustand, Material-UI, Axios, Leaflet, SheetJS, Turf.js (optional), idb (if client storage needed).
- **Backend:** Node.js, Express, better-sqlite3 or pg, xlsx (for backend parsing), cors, dotenv.

5. Module / Block-by-Block Description

5.1 Configuration Block

- **UI:** Card with editable fields for origin, meter conversions, alphabet string (30 chars). Grid levels are fixed but can be toggled.
- **Backend:** API to save/fetch config from database.
- **Validation:** Alphabet must have exactly 30 unique characters.

5.2 Branch Master Block

- **UI:**
  – Upload area (drag & drop).
  – Table with columns: ID, City, Lat, Lon, Pocket ID (read-only).
  – Inline edit / delete.
  – “Add new” button.
  – Export button.
- **Backend:**
  – Endpoints for CRUD operations.
  – On upload, parse file and compute Pocket IDs for each branch (using geometry module).
  – Store in database.

5.3 Pocket Calculator Block

- **UI:** Two numeric inputs + generate button; result display with copy.
- **Backend:** /api/pocket/encode accepts lat/lon, returns Pocket ID and indices.
- **Logic:** Same geometry functions used on both ends (shared code).

5.4 Reverse Lookup Block

- **UI:** Input for Pocket ID + submit; display center and corners.
- **Backend:** /api/pocket/decode accepts ID, returns center coordinates.

5.5 Automatic Nearest Branch Finder

- **UI:**
  – Quick search form (lat/lon) and/or click on map.
  – Results panel showing nearest branches (list).
  – Map automatically highlights those branches and draws a radius.
- **Backend:** /api/nearest accepts lat/lon, max distance (optional), number of results; returns sorted branches with distances.
- **Frontend:** On map click, call API and update markers/layers.

5.6 Map Block

- **UI:** Central map with:
  – Base layer switcher.
  – Layer control: grid overlay (level selector), branches, customer dots.
  – Click to drop customer dot → triggers nearest branch search.
  – Popups with branch info and Pocket ID.
  – Highlight cell from reverse lookup.
- **Logic:** Grid overlay drawn dynamically based on current viewport and selected level. Use Leaflet's L.rectangle or L.gridLayer.

5.7 Batch Processing Block

- **UI:** Upload area, progress indicator, download button.
- **Backend:** /api/batch/encode accepts file, processes in background (streaming or queued), returns job ID or directly downloadable file.

---

6. Data Model

6.1 Configuration (single row)

sql

```
CREATE TABLE config (
 id INTEGER PRIMARY KEY CHECK (id = 1),
 origin_lat REAL NOT NULL,
 origin_lon REAL NOT NULL,
 meters_per_deg_lat REAL NOT NULL,
 meters_per_deg_lon REAL NOT NULL,
 alphabet TEXT NOT NULL, -- comma-separated 30 chars
```

grid_levels TEXT -- *JSON array of active levels*

);

6.2 Branch

sql

CREATE TABLE branches (

id VARCHAR(20) PRIMARY KEY,

city VARCHAR(100),

lat REAL NOT NULL,

lon REAL NOT NULL,

pocket_id VARCHAR(50) -- *computed, can be indexed*

);

6.3 (Optional) Customer Dots – stored only in frontend state, not persisted.

---

7. API Endpoints

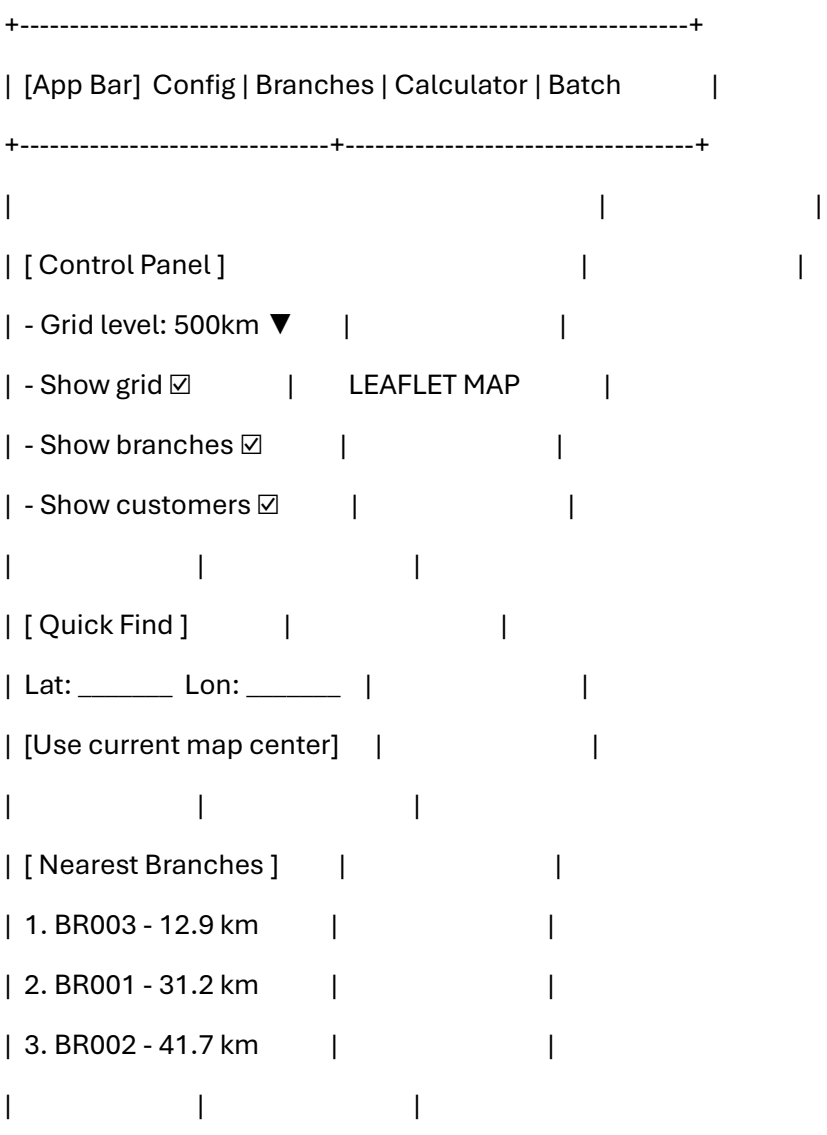
Method	Endpoint	Description
GET	/api/config	Retrieve current configuration
PUT	/api/config	Update configuration
GET	/api/branches	List all branches (with optional filters)
POST	/api/branches	Create a new branch
PUT	/api/branches/:id	Update branch
DELETE	/api/branches/:id	Delete branch
POST	/api/branches/upload	Upload Excel file (multipart/form-data)
GET	/api/branches/export	Export branches as Excel
POST	/api/pocket/encode	Body: { lat, lon } → returns { pocketId, indices }

Method	Endpoint	Description
POST	/api/pocket/decode	Body: { pocketId } → returns { centerLat, centerLon, corners }
POST	/api/nearest	Body: { lat, lon, maxDistance?, limit? } → returns array of branches with distance
POST	/api/batch/encode	Upload Excel file → returns download link

8. User Interface Design (Conceptual)

Professional layout with map as focal point:

text



+------------------------------+----------------------------------+

| [Branch Table Preview] (collapsible) |

+--+

- Map supports clicking to drop a customer dot and instantly update nearest list.
- All panels are draggable/stackable (using a library like react-grid-layout) if desired.

---

9. Development Phases (Production)

Phase 1: Backend Foundation

- Set up Node.js/Express project with database connection.
- Implement configuration API and database schema.
- Implement geometry module (conversion, indexing, encoding/decoding).
- Create branch CRUD API with Excel upload/export (using SheetJS on backend).

Phase 2: Frontend Foundation

- Set up React + Vite + TypeScript.
- Implement global state (Zustand) for config and branches.
- Build Config page with API integration.
- Build Branch management page with upload and table.

Phase 3: Pocket Calculator & Reverse Lookup

- Build UI for single coordinate encoding/decoding.
- Connect to backend APIs.

Phase 4: Nearest Branch Finder & Map Integration

- Integrate React-Leaflet with base tiles.
- Implement grid overlay drawing (dynamic based on config).
- Add branch markers with popups.
- Implement click-to-drop customer dot and call nearest API.
- Display nearest results list and highlight on map.
- Add radius circle.

Phase 5: Batch Processing & Polish

- Implement batch upload with progress.
- Add export functionality.
- Final UI polish, error handling, loading states.

- Write documentation and deployment guide.

---

10. Technical Stack (Updated)

Area	Technology
Frontend	React (Vite) + TypeScript
State Management	Zustand
UI Library	Material-UI (MUI)
Mapping	React-Leaflet + Leaflet
Excel Parsing	SheetJS (xlsx) – frontend & backend
Geospatial	Turf.js (frontend) or custom (backend)
Backend Framework	Node.js + Express
Database	PostgreSQL (prod), SQLite (dev)
API Communication	Axios
File Upload	Multer (backend)
Validation	Joi / Zod
Deployment	Docker, PM2, or static + Node hosting

11. Assumptions & Constraints

- **Assumption:** The grid is aligned with the origin; no projection.
- **Assumption:** Meters per degree longitude is constant across the region (based on origin latitude). For large countries, this may introduce small errors; future versions can incorporate variable scaling.
- **Constraint:** Browser must support modern JavaScript and Web Workers.
- **Constraint:** Map tiles require internet access (unless offline tiles are configured).

---

12. Future Enhancements

- User authentication and multi-tenant support.
- Real-time updates (WebSocket) for collaborative editing.

- Advanced geofencing: find all customers inside a given Pocket ID.
- Drill-down: click on grid cell to show branches inside.
- Support for custom map tile servers (e.g., satellite).
- Dark mode.

---

13. Deliverables

- Fully functional web application (frontend + backend) with source code.
- Database migration scripts.
- API documentation (Swagger/OpenAPI).
- User manual.
- Deployment guide for local and production environments.